

Vectorization of the Integer Calculations in the Graph Decomposition Problem

A. A. Rybakov*

(Submitted by A. M. Elizarov)

National Research Centre “Kurchatov Institute”, Moscow, 123182 Russia

Received July 17, 2025; revised September 21, 2025; accepted September 25, 2025

Abstract—The article is devoted to the problem of integer calculations vectorization. Integer calculations is a program context which mainly works with integers and contains a large number of control transfer operations. Such a program context is often found in combinatorial optimization problems where one has to work with discrete data structures, memory access indirection, and loop nests with an irregular number of iterations. These factors negatively affect the use of vectorization, and automatic vectorization is almost impossible for such a context. The article discusses the possibility of using the AVX-512 instructions set to vectorized code with the listed features, since the advantages of masked vector operations make it possible to achieve acceleration even in the presence of such bottlenecks. The application of vectorization is considered using the example of a bubble growth algorithm for graph decomposition, where the graph is represented as lists of vertex adjacencies. For the algorithm under consideration, a method is described for applying vectorization to loop nests using integer masked vector instructions from the AVX-512F set. The vectorized decomposition algorithm has been tested on graphs with up to 4 million vertices. Practical results were obtained on an Intel Xeon Phi 7290 Knights Landing microprocessor, and they demonstrated acceleration of the program code in the range of 1.7–2.2 times, depending on the size of the computational mesh.

2010 Mathematics Subject Classification: 68W10, 68R10.

DOI: 10.1134/S1995080225610987

Keywords and phrases: *vectorization, AVX-512, integer calculations, combinatorial optimization, graph decomposition, bubble growth algorithm*

1. INTRODUCTION

High-performance computing makes it possible to simulate natural and technological processes with a high degree of detail, simultaneously take into account many factors of various origins, and quickly analyze numerous options. This allows us to obtain a picture of the process under study with the required accuracy, reflecting the actual situation as much as possible instead of expensive, dangerous or even impossible full-scale experiments. The use of supercomputing technologies can significantly reduce the cost and time required to create new competitive products, facilities, and systems. The range of high-performance computing applications is very wide. Thus, using supercomputer modeling, problems of gas dynamics and hydrodynamics [1], molecular dynamics [2], plasma physics [3], solid mechanics [4], mathematics [5], and others are solved. High-performance computing is used to solve such large computational scientific and technical problems as atmospheric modeling [6], aircraft design [7], exploration of oil and gas fields, modeling of large molecules and polyatomic systems [8] and many others.

When solving a computational problem on a supercomputer system, parallelization must be performed between its nodes to speed up calculations [9]. The next level of computing parallelization is

*E-mail: rybakov.aax@gmail.com, rybakov_aan@nrcki.ru, rybakov@jscs.ru

parallelization within a single computing node on shared memory [10]. The lowest level of calculations parallelization can be considered vectorization—parallelization at the level of individual instructions [11].

Vectorization is an important low-level optimization of software code, which can be used to achieve multiple acceleration of supercomputer applications. All major modern microprocessor architectures (x86, ARM, Power, “Elbrus”, LoongArch, Sunway, and others) support vector computing, and there is a tendency to increase the size of the vector (today the maximum length is 512 bits, but now this length is logically unlimited, given the sets of vector instructions with a variable length of the vector).

Currently, the most advantageous set of vector instructions is the AVX-512, as it supports the possibility of selective processing of vector elements using vector masks. This unique feature allows to vectorize a complex programming context containing control transfer commands, loop nests, and function calls. It can be considered that for the first time the AVX-512 vector instructions set was supported in 2016 in Intel Xeon Phi Knights Landing (KNL) microprocessors, since the earlier generation Intel Xeon Phi Knights Corner (KNC) was an accelerator, and the vector code for it is not x86 compatible. Since that moment, the issues of application vectorization have been actively discussed in the scientific community. There are many papers on the vectorization of the software in application to various subject areas: gas dynamic solvers [12], encryption tools [13], time series analysis [14], sorting of an array of numbers [15], fast Fourier transform [16] and more.

In this paper, we will consider the application of vectorization using the AVX-512 instruction set to integer calculations containing loops with irregular number of iterations. As an example of such calculations, we will take the bubble growth algorithm of graph decomposition, in which a simultaneous breadth-first traversal of the graph is performed from randomly located initiating vertices.

2. BUBBLE GROWTH GRAPH DECOMPOSITION ALGORITHM

Currently, there is a wide variety of graph decomposition algorithms that are applied depending on the size of the graph being processed and the requirements for the quality of decomposition [17]. At the same time, the size requirements of the processed graphs are constantly increasing, and today the decomposition problem is considered for graphs containing tens of millions of vertices or more [18]. When performing graph processing, the main operation is to obtain incident edges and neighbouring vertices of a given vertex. The way this action is performed depends on the internal representation of the graph in the computer memory [19, 20]. We will consider a representation of an undirected graph in which a list of neighbouring vertices is available for each vertex index. Let's consider the following graph decomposition algorithm—the bubble growth algorithm—which is one of the simplest in terms of work logic.

The bubble growth algorithm for graph decomposition [21] is based on a breadth-first traversal of the graph, performed simultaneously, starting from several initiating vertices. At the beginning of the algorithm, the initiating vertices are randomly selected (according to the required number of domains). Each of the selected vertices belongs to a corresponding domain. After that, the process of simultaneous domains building begins. At each iteration of the algorithm, each domain is increased by one vertex that has not yet been assigned to any domain (if this can be done). The process ends when all the vertices of the graph are distributed across domains. Fig. 1 illustrates the process of building domains from randomly selected vertices for a graph on a plane.

After distributing all the vertices by domains, the center of domains are taken as new initiating vertices, the graph decomposition is reset, and then the domains are built from these new initiating vertices. This process can continue either a fixed number of times, or until a certain condition is reached (stabilization of the initiating vertices, achieving a balanced partition, repeating the previously obtained configuration) [22]. The bubble growth algorithm does not guarantee a balanced division into domains or the formation of short boundaries between domains. However, this algorithm is distinguished by its simplicity, which allows it to be used as a basic operation in complex approaches to the graph decomposition [23].

Let us briefly note the potential of using the bubble growth algorithm in the family of genetic algorithms [24]. Genetic algorithms are nature-like heuristic algorithms that are actively used to solve optimization problems with a large number of parameters [25]. During the operation of the genetic algorithm, the survival process of a population consisting of individuals is modeled, and each of them represents a solution for some optimization problem. To implement the algorithm, the concept of a

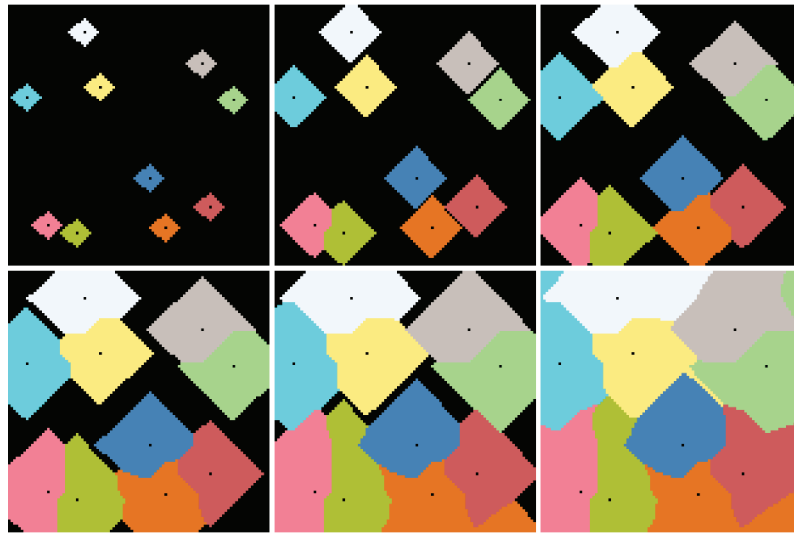


Fig. 1. Domains growth in the bubble decomposition algorithm.

genotype (some code of an individual) must be defined, the process of creating an individual based on its genotype must be implemented, the fitness function of the individual must be determined, and the operations of crossing and mutating individuals at the genotype level must be implemented. In the process of population evolution, the weakest individuals die out, while the more fit ones leave offspring, which leads to an increase in the average value of the fitness function of individuals in the population. Genetic algorithms are applicable for solving combinatorial optimization problems, in particular for graph decomposition problems. However, when using a genetic algorithm to solve the graph decomposition problem, the concepts of genotype and individual are often mixed. For example, in [26], when decomposing a graph, each edge is encoded in the genotype, and in [27], the explicit distribution of vertices across domains is encoded in the genotype. In other words, the genotype practically replaces the concept of an individual, which leads to a significant slowdown in the work of the genetic algorithm and the impossibility of its use for the decomposition of large graphs. Also, the application of the mechanisms of crossing and mutation directly to individuals is questionable from the point of view of correctness. The simplicity and speed of the bubble growth algorithm allows it to be used in genetic algorithms as an operation for constructing an individual from a genotype represented by a set of initiating vertices. In this case, the shift of the initiating vertex to a random neighbour can be considered as a genotype mutation. This approach of using the bubble growth algorithm inside the genetic algorithm has been approbated in practice, and the solution is available in an open repository <https://github.com/r-aax/mendel>.

3. BUBBLE GROWTH ALGORITHM VECTORIZATION

The use of the bubble growth algorithm as a single operation in complex approaches to graph decomposition (like the genetic algorithm) places increased demands on its performance. Let's consider the possibilities of speeding up this algorithm using AVX-512 instruction set for vectorization. Let's say we have a graph, information about its edges is written in the structure `vector<vector<int>> inc`, where `inc[i]` is a list of numbers of all vertices adjacent to the vertex `i` (adjacency list, or vertex neighbourhood). The domain numbers that specific vertices belong to are stored in the `vector<int> domains` structure. The `vector<queue<int>> q` structure is a list of queues. Each queue `q[i]` is queue of vertices waiting to enter the `i`-th domain. At the beginning of the algorithm, the `q[i]` queue contains only one initiating vertex of the `i`-th domain. Let's say we need to decompose the graph (or color the graph) into `domains_count` domains. Then, the simple implementation of the bubble growth algorithm for domains from the initiating vertices may be written as simultaneous breadth-first traversal of the graph starting from the initiating vertices (see Listing ??):

The implementation of the algorithm is performed as the nest of 3 loops. The outer loop is executed until there is at least one non-empty domain queue (that is, there are vertices not allocated by domains).

Listing 1. Implementation of the bubble domain growth algorithm.

1	<code>while (is_q) {</code>
2	<code>{</code>
3	<code>is_q = false;</code>
4	
5	<code>for (size_t c = 0; c < domains_count; ++c)</code>
6	<code>{</code>
7	<code>if (q[c].empty()) continue;</code>
8	
9	<code>is_q = true;</code>
10	<code>n = q[c].front();</code>
11	<code>q[c].pop();</code>
12	
13	<code>if (domains[n] == -1)</code>
14	<code>{</code>
15	<code>domains[n] = c;</code>
16	<code>for (auto ngh : inc[n]) q[c].push(ngh);</code>
17	<code>}</code>
18	<code>}</code>
19	<code>}</code>

The middle cycle is performed by domain numbers. For each domain, the first vertex is taken from the corresponding queue, and if it has not yet been assigned to any domain, it enters the current domain, and all its neighbours are sent to the queue. The inner loop is a loop through all the neighbours of the newly processed vertex that should be queued. We will perform vectorization of the presented code by the middle cycle, and for ease of analysis, we will provide an implementation for a fixed value `domains_count = 16` (this will get rid of the middle cycle and replace it with a set of vector operations).

To perform vectorization, first we need to get rid of the STL structures (**vector** and **queue**), since they have their own internal implementation, and vectorization of operations for working with them is impossible. Instead of the `vector<int>` structure, we will store information about the list of neighbouring vertices simply in an array, the 0-th element of which will be its size. The queue will also be simulated using an array and indexes **front** and **back**, pointing to the first and last elements of the queue, respectively. Then, the operation `push(v)` will correspond to writing into the array at the index **back** with its incrementation, and the operation `pop` will correspond simply to the incrementation of the index **front**. The queue is empty if its index **front** is greater than the index **back**.

On Fig. 2 the scalar and vector versions of the software code for implementing the bubble growth algorithm are presented. In the vector version, **ADD**, **GTH**, and **SCT** are used to denote the intrinsic functions `_mm512_mask_add_epi32`, `_mm512_mask_i32gather_epi32`, `_mm512_i32scatter_epi32` respectively (**GTH2** and **SCT2** are the same operations **GTH** and **SCT**, only using two offsets from the base address at once). Using **CMPLT**, **CMPLT**, the calls to the intrinsic function `_mm512_mask_cmp_epi32_mask` are indicated with the comparison parameters `_MM_CMPINT_LE` and `_MM_CMPINT_LT`, respectively. On Fig. 2 the digit “1” indicates the vectorization of the condition for continuing execution of the outer loop (the loop terminates if all domain queues are empty). The number “2” indicates the vectorization of extracting the next vertex from each queue. The number “3” indicates that the extracted vertices belong

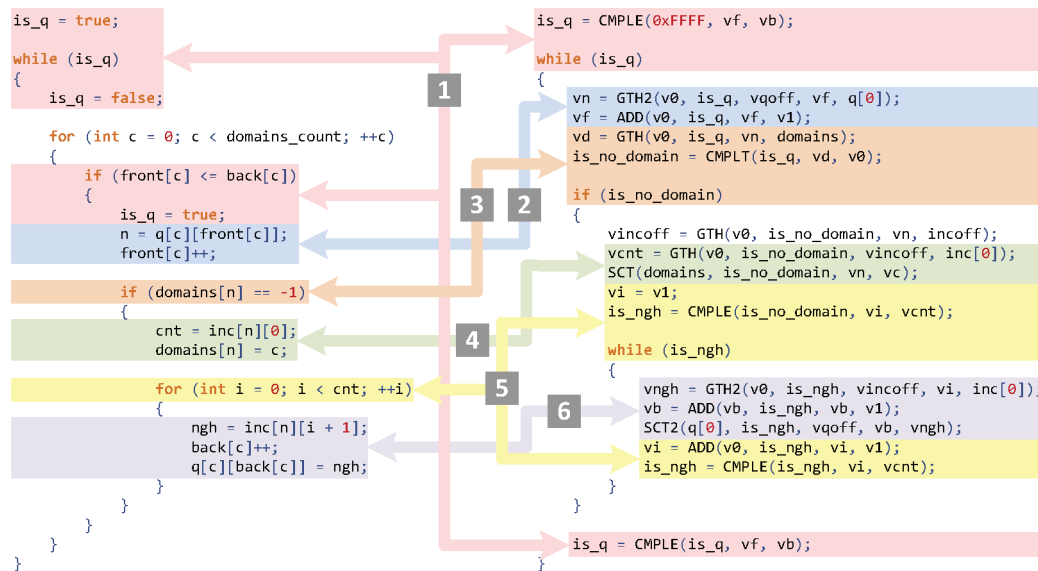


Fig. 2. Vectorization of the bubble growth algorithm program code by replacing scalar instructions with vector analogues.

to some domain (in the vector version, a mask `is_no_domain` is formed—a mask with the numbers of the domains to which the new vertex is added). The number “4” indicates the vectorization of placing the vertex into the current domain and obtaining the number of its neighbours. The digit “5” is processing all the neighbours of the vertex just placed in the domain, and the digit “6” is adding these neighbours to the corresponding queues.

It is worth noting that the vectorized version of the presented algorithm is not fully equivalent to the scalar version. It may turn out that at some iteration of the outer loop, the same vertex will be extracted from two or more different queues at the same time. In this case, in the vector version, it will be assigned to the domain with the highest number (although in the scalar equivalent, it would fall into the domain with the lowest number). This discrepancy can be eliminated by turning around the vectors `vn` and `vc` when writing to the array `domains` (digit “4” on Fig. 2), but this operation was not implemented for code simplification.

To evaluate the effectiveness of the vectorization, test runs were performed for dual graphs of surface rectangular computational meshes with sides from 20 to 2000 cells (that is, on graphs with a number of vertices from 400 to 4 million). On Fig. 3 on the left the collected statistics on the masks density (fraction of set bits) of various operations types is presented. From these charts, we can immediately conclude that vectorization is not very efficient, since the density of masks for arithmetic operations and write operations to memory turned out to be quite low (about 0.3 and 0.25, respectively).

Measurements of the actual acceleration were performed for the same graphs on Intel Xeon Phi 7290 Knights Landing microprocessor, the results are shown on Fig. 3 on the right. For convenience, there is also a chart of the smoothed acceleration indicator. Analyzing the chart of the smoothed acceleration indicator, it can be noted that the maximum is observed when the side of the computational mesh is around 750 and is approximately 1.95. The decrease in acceleration when reducing the side of the computational mesh is associated with an increase in the number of conflicts when processing the next vertices from the domain queues. The decrease in acceleration with an increase in the side of the computational mesh is associated with an increase in the spread of offsets when performing gather/scatter operations, which leads to cache misses.

4. CONCLUSIONS

The article reviewed the bubble growth algorithm used for graph decomposition. This algorithm is used to perform decomposition both as an independent method and as part of other, more complex approaches to decomposition. The implementation was considered in which each domain maintains

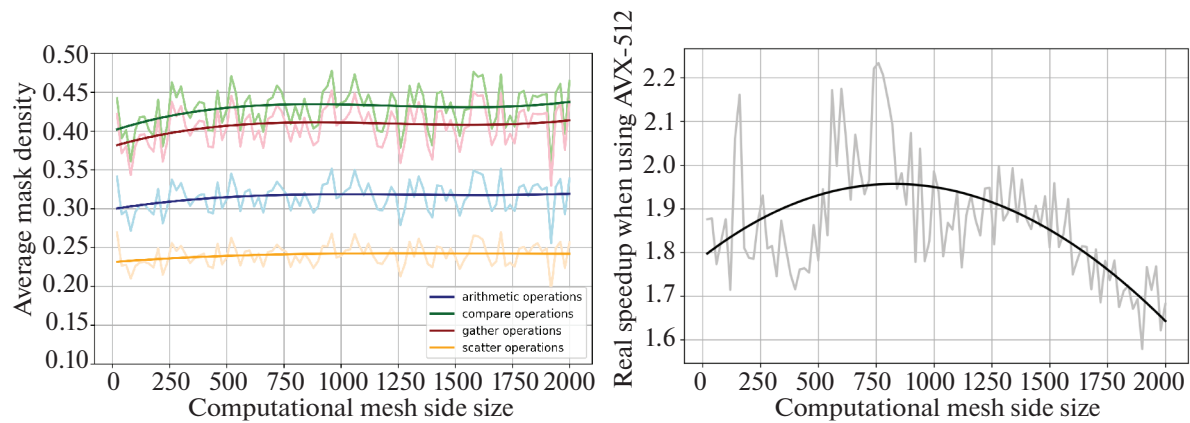


Fig. 3. The average density of masks for different AVX-512 operations classes (on the left) and the acceleration of calculations obtained on Intel Xeon Phi 7290 Knights Landing (on the right).

its own queue of processed graph vertices. This approach allows simultaneous processing of domain queues, which makes it possible to use vectorization to speed up calculations. A variant of the algorithm vectorization for a fixed number of domains equal to 16 was proposed. Vectorization is performed for the middle cycle by replacing scalar operations with vector analogues. For the obtained vectorization variant, statistics on the vector operations masks density in the resulting code were collected, which showed extremely low values for arithmetic operations (about 0.3) and write operations to memory (about 0.25). Such a low masks density is associated with the behavior of the inner loop, the number of iterations of which can be described as irregular, that is, it does not depending on the iteration number of the middle cycle. Acceleration measurements on the Intel Xeon Phi 7290 Knights Landing microprocessor demonstrated acceleration in the range of 1.7–2.2 times for the graphs under consideration with a number of vertices from 400 to 4 million. Such low acceleration results are primarily due to the irregular number of loop iterations in the nest, as well as the abundance of multiple access operations to memory (gather/scatter).

FUNDING

The work was carried out as part of the state assignment of the National Research Centre “Kurchatov Institute”.

CONFLICT OF INTEREST

The author declares no conflict of interest.

REFERENCES

1. E. M. Smirnov, D. K. Zaitsev, A. A. Smirnovsky, et al., “Assessment of several advanced numerical algorithms implemented in the CFD code SINF/Flag-S for supercomputer simulations,” *Supercomput. Front. Innov.* **11** (2), 14–31 (2024). <https://doi.org/10.14529/jsfi240202>
2. Z. Guo, D. Lu, Y. Yan, et al., “Extending the limit of molecular dynamics with ab initio accuracy to 10 billion atoms,” in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, April 2–6, 2022, Seoul, Republic of Korea* (2022). <https://doi.org/10.1145/3503221.3508425>
3. C. Asch, E. Franceschini, and E. Meneses, “An implementation of a plasma physics application for distributed-memory supercomputers using a directive-based programming framework,” *Rev. Colomb. Comput.* **25**, 39–47 (2024). <https://doi.org/10.29375/25392115.5053>
4. N. Morgan, C. Yanusah, A. Diaz, et al., “Enabling parallel performance and portability of solid mechanics simulations across CPU and GPU architectures,” *Information* **15**, 716 (2024). <https://doi.org/10.3390/info15110716>
5. R. Lohiya and N. Kumar, “Harnessing the power of supercomputers: Solving complex mathematical problems and queueing theory,” *Int. J. Res. Publ. Semin.* **15**, 166–172 (2024). <https://doi.org/10.36676/jrps.v15.il.1398>

6. J.-S. Kang, H. Myung, and J.-H. Yuk, "Examination of computational performance and potential applications of a global numerical weather prediction model MPAS using KISTI supercomputer NURION," *J. Mar. Sci. Eng.* **9**, 1147 (2021). <https://doi.org/10.3390/jmse9101147>
7. A. Morad, "Multidisciplinary conceptual investigation for integrating stores, not in the original configuration of a subsonic airplane," *J. Phys.: Conf. Ser.* **2616**, 012005 (2023). <https://doi.org/10.1088/1742-6596/2616/1/012005>
8. Y.-J. Yan, H.-B. Li, T. Zhao, et al., "10-million atoms simulation of first-principle package LS3DF," *J. Comput. Sci. Technol.* **39**, 45–62 (2024). <https://doi.org/10.1007/s11390-023-3011-6>
9. V. V. Voevodin, D. I. Shaikhislamov, and D. A. Nikitenko, "How to assess the quality of supercomputer resource usage," *Supercomput. Front. Innov.* **9** (3), 4–18 (2022). <https://doi.org/10.14529/jsfi220301>
10. Q.-W. Zhou, J.-N. Li, R.-C. Zhao, et al., "Compilation optimization of DCU-oriented OpenMP thread scheduling," *J. Phys.: Conf. Ser.* **2558**, 012003 (2023). <https://doi.org/10.1088/1742-6596/2558/1/012003>
11. J. Feng, Y. He, and Q. Tao, "Evaluation of compilers' capability of automatic vectorization based on source code analysis," *Sci. Programm.* **6**, 1–15 (2021). <https://doi.org/10.1155/2021/3264624>
12. I. Kulikov, I. Chernykh, and A. Tutukov, "A new hydrodynamic code with explicit vectorization instructions optimizations that is dedicated to the numerical simulation of astrophysical gas flow. I. Numerical method, tests, and model problem," *Astrophys. J. Suppl. Ser.* **243**, 4 (2019). <https://doi.org/10.3847/1538-4365/ab2237>
13. B. Buhrow, B. Gilbert, and C. Haider, "Parallel modular multiplication using 512-bit advanced vector instructions," *J. Cryptogr. Eng.* **12**, 95–105 (2022). <https://doi.org/10.1007/s13389-021-00256-9>
14. R. Quislan and I. Fernandez, "Time series analysis acceleration with advanced vectorization extensions," *J. Supercomput.* **79**, 10178–10207 (2023). <https://doi.org/10.1007/s11227-023-05060-2>
15. M. Blacher, J. Giesen, P. Sanders, et al., "Vectorized and performance-portable Quicksort," *arXiv: 2205.05982* (2022). <https://doi.org/10.48550/arXiv.2205.05982>
16. G. Sansone and M. Cococcioni, "Experiments on speeding up the recursive fast Fourier transform by using AVX-512 SIMD instructions," in *Applications in Electronics Pervading Industry, Environment and Society, ApplePies 2022*, Ed. by R. Berta and A. De Gloria, *Lect. Notes Electr. Eng.* **1036**, 255 (2023). https://doi.org/10.1007/978-3-031-30333-3_34
17. T. Ayall, H. Liu, C. Zhou, et al., "Graph computing systems and partitioning techniques: A survey," *IEEE Access* **10**, 118523–118550 (2022). <https://doi.org/10.1109/ACCESS.2022.3219422>
18. H. Lee, J. Baek, S. Song, et al., "Efficient large graph partitioning scheme using incremental processing in GPU," *IEEE Access* **13**, 43889–43903 (2025). <https://doi.org/10.1109/ACCESS.2025.3547976>
19. A. Ahmed, F. Siddique, K. Skadron, et al., "GraphTango: A hybrid representation format for efficient streaming graph updates and analysis," *Int. J. Parallel Program.* **52**, 147–170 (2024). <https://doi.org/10.1007/s10766-024-00768-x>
20. D. Salwasser, D. Seemaier, L. Gottesbüren, et al., "Tera-scale multilevel graph partitioning," *arXiv: 2410.19119* (2024). <https://doi.org/10.48550/arXiv.2410.19119>
21. W. Fan, M. Liu, C. Tian, et al., "Incrementalization of graph partitioning algorithms," *Proc. VLDB Endowm.* **13**, 1261–1274 (2020). <https://doi.org/10.14778/3389133.3389142>
22. E. N. Golovchenko, "Overview of graph decomposition algorithms," *KIAM Preprint No. 2* (Keldysh Inst. Appl. Math., Moscow, 2020). <https://doi.org/10.20948/prepr-2020-2>
23. Y. Wu, J. Du, and D. Ni, "Balanced domain partitioning for software defined networks," *IEEE Access* **11**, 6467–6476 (2023). <https://doi.org/10.1109/ACCESS.2023.3237733>
24. S. Katosh, S. Chauhan, and V. Kumar, "A review on genetic algorithm: Past, present and future," *Multimedia Tools Appl.* **80**, 8091–8126 (2020). <https://doi.org/10.1007/s11042-020-10139-6>
25. N. Wirayanti and H. Sriwindono, "Implementation of hybrid genetic algorithm for solving the teacher placement problem," *Soc. Sci. Human. J.* **9**, 6341–6347 (2025). <https://doi.org/10.18535/sshj.v9i01.1460>
26. A. Chaouche and M. Boulif, "Edge-set reduction to efficiently solve the graph partitioning problem with the genetic algorithm," *arXiv: 2307.10410* (2023). <https://doi.org/10.48550/arXiv.2307.10410>
27. M. Li, H. Cui, C. Zhou, et al., "GAP: Genetic algorithm based large-scale graph partition in heterogeneous cluster," *IEEE Access* **8**, 144197–144204 (2020). <https://doi.org/10.1109/ACCESS.2020.3014351>

Publisher's Note. Pleiades Publishing remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

AI tools may have been used in the translation or editing of this article.